A continuación, se presenta una reorganización explicativa de los componentes técnicos esenciales utilizados para la gestión y conexión segura de bases de datos:

1. PHP Data Objects (PDO) En versiones anteriores de PHP, la comunicación con las bases de datos se realizaba mediante funciones tradicionales como mysqli_query, las cuales presentaban importantes vulnerabilidades de seguridad. En la actualidad, PDO se erige como una clase moderna y orientada a objetos que opera como un traductor universal entre el código y el motor de datos. Su principal beneficio radica en la protección automática que brinda contra ataques de "SQL Injection" (uno de los métodos de hackeo más extendidos a nivel global) a través de la implementación de "Sentencias Preparadas". Cuando se ejecuta la instrucción $this->pdo = new PDO(...), se está inicializando este traductor y suministrándole las credenciales de acceso a MySQL.

2. Data Source Name (DSN) El DSN ($dsn) consiste en una cadena de texto (String) que establece los parámetros de localización o coordenadas esenciales para PDO. Su función es especificar las directrices de conexión, tales como el uso de una base de datos MySQL, el host correspondiente, el nombre de la base y el tipo de codificación del texto.

3. La Constante Mágica **DIR** En el ecosistema de PHP, **DIR** es identificada como una "constante mágica". Cuenta con la propiedad de reconocer con total exactitud la carpeta en la que se encuentra almacenado el archivo en ejecución, lo que facilita la localización de rutas relativas de manera independiente a la computadora donde se implemente el sistema.

4. Gestión de Errores: try { ... } catch { ... } y PDOException Para garantizar la estabilidad de la aplicación, se estructuran los siguientes mecanismos de contingencia: Estructura Try-Catch: Funciona como una red de seguridad operativa. La instrucción determina que el sistema intente (try) procesar un bloque de código; en caso de que ocurra una falla crítica, el sistema captura (catch) la anomalía para evitar el despliegue de una pantalla de error incomprensible para el usuario, permitiendo mostrar un mensaje amigable en su lugar. PDOException: Representa la alarma específica del sistema de datos. Si PDO no logra establecer el enlace debido a que MySQL se encuentra inactivo o a que los datos de acceso son erróneos, se genera e interrumpe el flujo mediante una alerta denominada PDOException.

Archivo: modelos/Conexion.php Su propósito fundamental es establecer un enlace técnico entre la lógica del sistema y el motor de MySQL bajo estrictos protocolos de integridad.

En cuanto a la infraestructura de comunicación, se emplea PDO (PHP Data Objects). Esta herramienta se desempeña como un mediador universal que resguarda la aplicación de forma nativa frente a vectores de ataque por inyección SQL. Para optimizar los recursos del servidor, se implementa el Patrón Singleton (private static $instancia). Mediante la privacidad del método __construct(), se impide la generación de conexiones redundantes. La lógica se centraliza en getInstance(), función encargada de verificar la existencia previa del enlace para su reutilización o creación exclusiva. La estabilidad se gestiona mediante el bloque try { ... } catch { ... }, que opera como una red de seguridad operativa. Este mecanismo permite que, ante una falla crítica de MySQL, el sistema capture (catch) la anomalía y despliegue una respuesta controlada en lugar de interrumpir la experiencia del usuario con errores técnicos.

5. Archivo: publico/test_bd.php Consiste en un módulo de diagnóstico temporal diseñado para validar la integridad de las fases iniciales de configuración.

El procedimiento consistió en la integración de la clase Conexion mediante las instrucciones require_once y el espacio de nombres correspondiente. Tras solicitar la instancia de datos, se verificaba el estado de Apache y

MySQL en el entorno XAMPP a través de una confirmación visual en pantalla.

6. Archivo: ayudantes/Sesion.php Actúa como el guardián de seguridad del sistema, estableciendo una barrera perimetral para restringir el acceso a los módulos administrativos según el estado de autenticación.

Dada la naturaleza del protocolo HTTP, PHP utiliza la Superglobal $_SESSION. Este repositorio temporal en el servidor almacena la información vital del usuario, como su identidad y privilegios, tras una validación exitosa de credenciales. Debido a que las funciones se definieron como static, se emplea la referencia self:: para la invocación de herramientas internas de la clase, eliminando la necesidad de instanciar nuevos objetos. Controles de Ciberseguridad (OWASP): session_regenerate_id(true): Actualiza el identificador de sesión durante el acceso para mitigar riesgos de Session Fixation. Control de Expiración (time() + 1800): Implementa una política de Timeboxing que finaliza la sesión tras 30 minutos de inactividad. Depuración de Cookies: Durante el cierre de sesión, se eliminan los registros locales en el navegador para garantizar la máxima protección de los datos.

7. Archivo: modelos/Usuario.php Su propósito fundamental es desempeñarse como el mediador técnico (Modelo de Datos) encargado exclusivamente de la comunicación con la tabla de usuarios en MySQL.

Bajo una Arquitectura Orientada a Objetos, se define una clase cuya responsabilidad única es la interacción con la información. Mediante el método __construct(), se inicializa el enlace al solicitar la instancia a la clase Conexion, garantizando que al instanciar new Usuario(), la conexión al motor de datos se establezca de forma automática. Para garantizar la Protección contra Inyección SQL, el método buscarPorEmail implementa el uso de "Marcadores" (:email). Esta técnica evita la concatenación directa de variables en la instrucción SQL, mitigando uno de los vectores de ataque más críticos en el desarrollo web. El flujo operativo de PDO (prepare -> execute -> fetch): prepare(): Transmite la estructura de la consulta a MySQL para su compilación segura y preventiva. execute(): Realiza la sustitución técnica del marcador por el valor real proporcionado por el usuario. fetch(PDO::FETCH_ASSOC): Recupera la información obtenida y la transforma en un Arreglo Asociativo, facilitando su lectura mediante claves descriptivas como $usuario['contrasena_hash'].

8. Archivo: controladores/AuthControlador.php Objetivo: Recibir el formulario de login, procesarlo de forma segura y decidir a qué pantalla redirigir al usuario.

El patrón Controlador: Es el director de orquesta. Primero atrapo los datos que el usuario escribió ($_POST). Luego, llamo al Modelo de Datos (buscarPorEmail) para pedirle información a la base de datos. Finalmente, si todo está correcto, llamo a mi Clase de Seguridad (Sesion::iniciarLogin) y le digo al navegador hacia dónde ir (header("Location: ..."")). $_SERVER['REQUEST_METHOD']: Lo usamos para distinguir si el usuario recién entró a la página por primera vez (método GET, entonces le mostramos el formulario), o si acaba de hacer clic en el botón "Ingresar" (método POST, entonces procesamos sus datos). Función password_verify(): Una genialidad de PHP. Como en la base de datos guardamos las contraseñas encriptadas con BCRYPT (nunca se guardan en texto plano por seguridad), esta función compara mágicamente el texto que escribió el usuario (ej: "MiClave123") con el garabato inentendible guardado en la BD (ej: "$2y$10$e8wYVjFw6rNKGZqfD7h..."). Si el algoritmo matemático coincide, devuelve true. Manejo de Errores Vía Interfaz: Si el email o la contraseña están mal, creamos una variable $error = "..." y cargamos nuevamente el HTML del login (require). El archivo HTML tomará esa variable y dibujará un cartelito rojo en la pantalla.

9. Archivo: publico/index.php (Front Controller) Objetivo: Ser el único punto de entrada de toda la aplicación por motivos de seguridad y ruteo centralizado.

Seguridad Front Controller: Evita que los usuarios naveguen por carpetas internas (como /modelos/ o /controladores/). Todo el mundo entra por publico/index.php. Carga Centralizada: Es el mejor lugar para importar la configuración global y las herramientas que usamos siempre (como la Base de Datos o la Sesión). Ruteo Dinámico (Routing): En lugar de tener 50 archivos distintos (login.php, registro.php, perfil.php), usamos variables $_GET en la URL. Ejemplo: ?c=auth&a=mostrarLogin significa Controlador = AuthControlador, Acción = mostrarLogin(). Clases Variables: PHP tiene un superpoder que nos permite instanciar clases usando textos dinámicos. Primero armamos el texto con el nombre de la clase ($claseCompleta = "App\Controladores\AuthControlador"😉 y luego creamos el objeto escribiendo new $claseCompleta();.

10. Archivo: controladores/AdminControlador.php Su finalidad principal es salvaguardar el perímetro del área administrativa y organizar la información que se proyectará en el tablero de control (Dashboard).

Control de Acceso (__construct): Esta clase prioriza la ejecución de su constructor de forma inmediata. Al integrar Sesion::requerirLogin(), se establece una restricción técnica: si un usuario intenta vulnerar la ruta directa sin una autenticación previa, el sistema interrumpe el acceso y lo redirige al módulo de acceso. El método dashboard(): Representa el núcleo operativo del controlador. En fases posteriores, se encargará de realizar las consultas SQL para cuantificar usuarios y torneos vigentes, suministrando estos indicadores a la interfaz dashboard.php.

11. Concepto: Adaptación de Vistas y Rutas Absolutas (URL_BASE) Garantiza la correcta carga de activos digitales (estilos CSS, imágenes y scripts) con total independencia del controlador que invoque la interfaz visual.

Conflicto de Rutas Relativas (../): Bajo el patrón MVC, la interpretación del navegador se centraliza en publico/index.php, perdiendo la noción de la jerarquía física de carpetas que se utilizaba en maquetación estática. La solución estratégica: Se implementa la constante URL_BASE para establecer una referencia absoluta. Esto obliga al sistema a localizar los recursos desde el directorio raíz del proyecto de forma unificada. Arquitectura Limpia (CSS vs PHP): Se prohíbe el uso de estilos en línea para preservar la separación de responsabilidades. La lógica de presentación se confina a archivos .css, mientras que el .php se reserva exclusivamente para la lógica de datos. 12. Actualización: ayudantes/Sesion.php Optimización del repositorio temporal del servidor para almacenar metadatos adicionales tras una validación de identidad exitosa.

Se identificó la necesidad técnica de personalizar la interfaz del Dashboard. Para ello, se expandió la captura de información durante el proceso de autenticación inicial. Se incorporó la asignación de $_SESSION['usuario_nombre'] dentro de la lógica de iniciarLogin(), vinculándola directamente con los registros de la base de datos. Gestión de Persistencia de Sesión: Cualquier modificación estructural en el manejo de sesiones requiere la invalidación de los tokens activos. Es imperativo ejecutar un logout para forzar una nueva carga de variables en la memoria de PHP.

13. Refactorización del Controlador (Clean Code) y Arreglos Asociativos Objetivo: Mantener el AdminControlador limpio y estructurar los datos que se envían a la vista.

Funciones Privadas: En lugar de tener una sola función gigante que haga todo, creamos funciones de apoyo privadas (private function obtenerUsuarios()). Estas funciones solo pueden ser usadas por el propio controlador y ayudan a que el código sea modular, fácil de leer y de mantener. El Arreglo $datos: En el patrón MVC, es una excelente práctica agrupar toda la información que el controlador extrajo de la base de datos dentro de un único arreglo (ej: $datos = ['totalUsuarios' => X, 'totalTorneos' => Y]). De esta forma, le entregamos a la Vista un solo "paquete" con todo lo que necesita imprimir.

14. Modelo: Torneo.php Objetivo: Extender la capacidad de nuestro sistema para interactuar con la tabla de torneos en MySQL.

Estandarización: Al igual que el modelo Usuario, le aplicamos el patrón Singleton en el constructor (Conexion::getInstance()->getBD()) para asegurar el rendimiento. Filtros SQL: Implementamos el método obtenerTodos() usando la instrucción COUNT(*) acompañada de una cláusula WHERE para contar exclusivamente los torneos que están en estado activo o de inscripciones abiertas, descartando los cancelados o finalizados. 15. Arquitectura de Vistas (Plantillas Anidadas) Objetivo: Reemplazar el viejo enrutador falso de Javascript por inyección directa de PHP.

Separación Cascarón / Contenido: Mantenemos la estructura profesional de las maquetas donde dashboard.php funciona como el cascarón (contiene el , los estilos, el menú lateral y el pie de página). Inyección Dinámica (require_once): Dentro del cascarón, utilizamos PHP para inyectar la vista interna (inicio.php que contiene las tarjetas) usando require_once **DIR** . '/inicio.php';. Esto garantiza que PHP arme el rompecabezas completo en el servidor antes de enviárselo al navegador, manteniendo la seguridad de la sesión intacta.

16. Integración de Tablas Relacionales (SQL Avanzado) Objetivo: Obtener métricas complejas cruzando información de múltiples tablas en el Modelo Equipo.php.

Uso de LEFT JOIN y GROUP BY: Para poder contar cuántos miembros tiene un equipo, abandonamos las consultas simples de una sola tabla. Utilizamos un LEFT JOIN para fusionar la tabla equipos con equipo_miembros, y luego agrupamos (GROUP BY) los resultados por ID de equipo. Finalmente, la función COUNT(em.usuario_id) se encargó de calcular el total de integrantes por fila de manera dinámica y altamente eficiente.

17. Inyección de Datos Dinámicos en Vistas (Bucles foreach) Objetivo: Reemplazar el código HTML estático de las tablas del Dashboard por filas generadas dinámicamente según los registros de la base de datos.

Procesamiento de Arreglos: El Controlador recupera los registros utilizando el método fetchAll(PDO::FETCH_ASSOC), obteniendo un arreglo multidimensional. Renderizado con foreach: En la vista (inicio.php), se emplea la estructura de control foreach para iterar sobre el arreglo. Esta técnica instruye a PHP para generar una fila HTML () por cada registro iterado. Manejo de Estados Vacíos: Se implementa la validación empty() para asegurar que, ante la ausencia de registros en la base de datos, la interfaz mantenga su integridad estructural y presente un mensaje informativo al usuario.

18. Control de Acceso Basado en Roles (RBAC) y Seguridad Objetivo: Restringir el acceso a rutas administrativas, garantizando que usuarios con privilegios inferiores (Ej: Jugadores) no puedan ingresar a módulos críticos.

Implementación en la Clase de Seguridad (Sesion.php): Se desarrolló el método estático validarAdmin(), el cual ejecuta una validación de autorización estricta. Se verifica que el rol_id en la variable superglobal $_SESSION corresponda al valor 1 (Administrador General). De no cumplirse esta condición, el sistema interrumpe la carga de la página mediante exit y redirige la petición hacia el módulo de acceso con un parámetro de denegación. Aplicación en el Controlador (AdminControlador.php): El método constructor (__construct) fue actualizado para invocar Sesion::validarAdmin() previo a cualquier instanciación de modelos o renderizado de vistas, garantizando el blindaje automático de la clase.

19. Enrutamiento MVC desde la Interfaz de Usuario Objetivo: Interconectar el menú lateral con el Front Controller (index.php) para habilitar la navegación dinámica entre módulos bajo el patrón MVC.

Refactorización de Enlaces: Se sustituyeron los anclajes HTML estáticos por rutas absolutas estructuradas (ej: href="index.php?c=usuario&a=index"). Esta nomenclatura instruye al enrutador central para instanciar el Controlador pertinente y ejecutar la acción requerida. Habilitación de Controladores: Se reacondicionó el archivo UsuarioControlador.php, implementando su respectivo constructor con validación de roles y un método index() como punto de entrada para la gestión de usuarios.

20. Arquitectura de Interfaz y Experiencia de Usuario (UX) Objetivo: Alinear la estructura de navegación del Administrador General con los requisitos funcionales documentados en las especificaciones del proyecto.

Reestructuración Lógica del Menú: Se diseñó una nueva jerarquía de navegación basada en tres bloques funcionales:

1. Gestión Humana: Centraliza la administración de todas las entidades (Administrativos, Organizadores, Jugadores y Equipos).
2. Gestión de Competencias: Agrupa las facultades operativas de nivel superior (Catálogo de Juegos, Supervisión de Torneos, Registro Global de Resultados).
3. Configuración del Sistema: Aísla las funciones de mantenimiento y auditoría (Roles, Alertas Globales, Logs). Justificación de Diseño: Esta categorización optimiza la usabilidad (UX) del software, disminuye la carga cognitiva durante la navegación y establece bases para una programación modular de futuros Controladores.
4. Patrón de Plantilla Maestra (Inyección Dinámica de Vistas) Objetivo: Mitigar la duplicación de código HTML estructural (cabeceras, menús y pies de página) mediante el uso de un contenedor maestro reutilizable.

Resolución Arquitectónica: Se refactorizó el archivo dashboard.php para cargar vistas internas de forma dinámica a través de la variable $vistaInyectada (ej: require_once **DIR** . '/' . $vistaInyectada;). Responsabilidad del Controlador: El Controlador asume la responsabilidad de definir el valor de $vistaInyectada previo a la invocación de la plantilla. Esta modificación permite un ensamblaje dinámico y centralizado de las vistas en el servidor.

22. Pantalla de Gestión de Usuarios (usuarios_index.php) Objetivo: Implementar una interfaz administrativa que exponga el directorio completo de cuentas registradas en el sistema.

Modelo de Datos (INNER JOIN): Se desarrolló el método obtenerTodosUsuariosConRoles() en la clase Usuario. La consulta implementa un INNER JOIN con la tabla roles, permitiendo vincular la llave foránea (rol_id) con la descripción nominal del rol, optimizando así la lectura de los datos. Vista y Lógica Condicional: Se integró lógica de presentación dentro de las iteraciones de la tabla. Mediante estructuras condicionales en PHP, la interfaz renderiza componentes visuales diferenciados (etiquetas de "Activo" o "Bloqueado") basándose en el estado booleano almacenado en la base de datos.

23. Patrón de Diseño: Strategy (Arquitectura de Algoritmos) Objetivo: Encapsular algoritmos complejos y variables (como la generación de fixtures y cruces) fuera de la clase principal, permitiendo su intercambio dinámico sin alterar el núcleo del sistema.

Planteamiento del Problema: Si la clase Torneo debiera gestionar internamente la generación de fixtures para modalidades de Liga, Eliminación Directa y Suizo, se generaría un bloque de código monolítico e insostenible

(violación del principio Open/Closed de SOLID). Solución Arquitectónica (Strategy): Se delegó la responsabilidad algorítmica a clases independientes (ModuloLiga, ModuloEliminacion, ModuloSuizo). La clase Torneo interactúa con estas clases como "estrategias" conectables. Beneficio Operativo: Alta escalabilidad. La integración de futuros formatos de competición (ej. Doble Eliminación o Fase de Grupos) solo requiere el desarrollo de una nueva clase módulo, preservando la integridad y estabilidad del código base.

24. Patrón de Diseño: Singleton (Gestión de Instancias) Objetivo: Garantizar que durante todo el ciclo de ejecución de la aplicación, exista una única instancia global para la conexión a la base de datos, previniendo el agotamiento de recursos del servidor (Too Many Connections).

Implementación en Conexion.php: Se privatizó el método constructor (__construct()) para bloquear la instanciación externa (new Conexion()). Controlador de Instancia: Se centralizó el acceso a través del método estático getInstance(). Este método verifica la existencia previa de la conexión; si no existe, la crea, y si ya existe, retorna la referencia activa. Beneficio Operativo: Optimización de recursos y mitigación del riesgo de colapso de MySQL ante solicitudes simultáneas concurrentes.

25. Centralizacion de Controladores y Redireccion Dinamica (Gestion de Usuarios) Objetivo: Evitar la duplicacion de codigo en operaciones comunes (editar perfiles, cambiar estado) manteniendo interfaces especializadas por rol.

Patron de Controlador Multiproposito: Se refactorizo UsuarioControlador.php para actuar como manejador central de operaciones CRUD. Aunque las vistas y listados esten divididos por sub-controladores (JugadorControlador, OrganizadorControlador), todas las acciones de guardado y edicion convergen en un unico punto. Redireccion Dinamica basada en Roles: Para mantener un flujo de UX transparente, el metodo actualizarUsuario() inspecciona la variable $datosActualizar['rol_id'] y utiliza esta informacion para redirigir (con header) al administrador de vuelta a la tabla especifica (Jugadores u Organizadores) desde la cual se origino la peticion.

26. Suplantacion de Identidad (Impersonation) Objetivo: Otorgar al Administrador la capacidad temporal de operar bajo los permisos y la vista de otro usuario registrado para facilitar soporte tecnico y auditoria.

Gestion Avanzada de Sesiones: Se diseno el metodo verComo() dentro de UsuarioControlador. Este metodo respalda el ID original del administrador en $_SESSION['admin_antes_suplantar'] antes de sobreescribir las variables criticas de sesion con los datos del usuario objetivo. Responsabilidad Arquitectonica: Al ubicar esta funcion en UsuarioControlador en lugar de un sub-controlador especifico, se respeta el Principio de Responsabilidad Unica (SRP), permitiendo su reutilizacion universal para cualquier rol del sistema sin generar acoplamiento.

27. Patron Flash Messages (Notificaciones Toast Hibridas) Objetivo: Proveer retroalimentacion visual no bloqueante (Toast Notifications) al usuario tras completar operaciones de backend, prescindiendo de AJAX.

Interconexion PHP-Javascript: Se implemento un flujo hibrido. El Controlador define $_SESSION['mensaje'] y ordena una redireccion. La vista maestra (dashboard.php) actua como listener: si detecta la variable en memoria, inyecta dinamicamente un bloque <script> que dispara la funcion frontend mostrarNotificacionToast(), procediendo inmediatamente a destruir la variable (unset) para garantizar que el mensaje sea efimero (Flash Message).

28. Bajas Fisicas vs Logicas (Modulo Usuarios) Objetivo: Completar el CRUD de usuarios con una eliminacion definitiva de la base de datos, distinguiendola de las suspensiones temporales.

Baja Logica (Bloquear): Implementada anteriormente cambiando el estado activo a 0. Preserva la integridad referencial y las estadisticas historicas (ej. torneos jugados). Baja Fisica (Eliminar Registro): Se implemento el metodo eliminarUsuario() usando la instruccion DELETE FROM con parametros vinculados (PDO). Esta accion destructiva se reservo exclusivamente para el Rol 1 (Administrador Supremo) y se centralizo en UsuarioControlador.php para poder eliminar indistintamente a jugadores, organizadores o miembros del staff.

29. Módulo de Catálogo de Juegos (Disciplinas) Objetivo: Permitir a los administradores gestionar la oferta deportiva y de eSports de la plataforma.

Arquitectura Base (CRUD Completo): Se construyó el modelo Juego.php y JuegoControlador.php implementando todas las operaciones CRUD (Crear, Leer, Actualizar, Borrar Lógicamente). Valores por Defecto para Simplificación: En el método crearJuego() del modelo, se decidió fijar (quemar) los valores de los puntos (3 para victoria, 1 para empate, 0 para derrota) y el estado activo directamente en la consulta SQL. Esto redujo drásticamente la complejidad del formulario frontend (juegos_crear.php), mejorando la usabilidad al pedirle al usuario únicamente 3 datos clave: Nombre, Categoría y Formato de Equipo. Baja Lógica (Switch de Estado): Se implementó el método cambiarEstado() utilizando un operador ternario para alternar entre 1 y 0. Esto permite "apagar" un juego para que no se puedan crear nuevos torneos con él, pero preservando el historial de torneos pasados (integridad de datos). Edición Dinámica: Se desarrollaron los métodos obtenerJuegoPorId() y actualizarJuego(). En la vista de edición (juegos_editar.php), se utilizaron condicionales ternarios en línea (`$juego['categoria'] === 'esport' ? 'selected' : ''`) para auto-completar los selectores desplegables `<select>` con los datos previos del registro.

30. Módulo de Políticas de Seguridad y Contraseñas (politicas_contrasenas - ADM-07) Objetivo: Permitir al Administrador General parametrizar dinámicamente las reglas de complejidad, expiración y reutilización de contraseñas para todos los usuarios del sistema.

Arquitectura MVC e Infraestructura:

- Modelo (PoliticaContrasena.php): Administra la lectura y persistencia de la fila activa (id = 1) en la tabla `politicas_contrasenas`. Implementa el método `obtenerPoliticaVigente()`, el cual auto-inicializa los valores por defecto si la tabla estuviera vacía.
- Controlador (PoliticaContrasenaControlador.php): Protegido con `Sesion::validarAdmin()`. La acción `guardar()` sanitiza los parámetros POST (longitud mínima, días de expiración, historial), actualiza el registro en la BD asociando el ID del administrador actualizador (`actualizado_por`), dispara la notificación Toast y registra la auditoría en `logs_actividad` mediante `Auditoria::registrar()`.
- Vista Administrable (politicas_contrasenas.php): Interfaz responsiva integrada al layout `dashboard.php` que expone campos numéricos, checkboxes de complejidad y metadatos de última modificación.

Funcionamiento y Lógica de Validación del Algoritmo (`validarPassword()`):

- Evaluación Independiente de Reglas: La regla de longitud mínima (`longitud_minima`) es totalmente independiente de los tipos de caracteres seleccionados (mayúsculas, minúsculas, números, caracteres especiales).
- Criterio de Presencia Mínima: Al activar los 4 requisitos de tipo de carácter con una longitud mínima de 10, el algoritmo NO exige una distribución equitativa o porcentual por cada tipo. Simplemente requiere que la contraseña cumpla con la longitud total global y que posea AL MENOS 1 carácter del tipo

solicitado. Por ejemplo, una clave como `Aa1!aaaaaa` es 100% válida (cumple los 10 caracteres y contiene al menos 1 mayúscula, 1 minúscula, 1 número y 1 especial, siendo los caracteres restantes libres).

- Alineación con Estándares Internacionales (OWASP / NSI): Esta lógica independiente preserva la usabilidad del sistema evitando confusiones al usuario y promoviendo el uso de frases de contraseña largas (passphrases), altamente seguras por su longitud.

31. Refactorización Arquitectónica: Restauración de Sesiones y Clean Code (Impersonation) Objetivo: Perfeccionar la función "Ver Como" asegurando una vía de retorno segura al rol de Administrador, sin violar principios arquitectónicos ni desencadenar bloqueos de seguridad por roles cruzados.

Inyección Global Limpia (Front Controller): En lugar de ensuciar el Front Controller (index.php) imprimiendo etiquetas <script> gigantes con toda la lógica del banner rojo flotante, se aplicó Clean Code. index.php se limitó a crear una variable global (window.URL_RESTAURAR_SESION) y cargar un archivo dedicado (publico/js/admin/suplantacion.js), preservando la separación de responsabilidades (Backend vs Frontend). Resolución de Conflictos RBAC (Constructor vs Métodos): Al intentar volver a la sesión de administrador usando UsuarioControlador->restaurarSesion(), el sistema generaba un Error 404 encubierto. Esto ocurría porque el método __construct() ejecutaba Sesion::validarAdmin() globalmente. Como el usuario estaba temporalmente con rol de Jugador/Organizador, el constructor lo bloqueaba antes de ejecutar la acción. La solución arquitectónica consistió en retirar el guardia del constructor y colocar Sesion::validarAdmin() de forma individual en cada método sensible, dejando restaurarSesion() de acceso libre para quien posea el token de retorno. Inicialización Explícita de Sesión: Al retirar el guardia global, PHP perdía la orden de arrancar la sesión (session_start()) para esa ruta específica. Se inyectó explícitamente Sesion::requerirLogin() al inicio de restaurarSesion() garantizando que la superglobal $_SESSION se construya en memoria y pueda validar correctamente $_SESSION['admin_antes_suplantar'] antes de devolver los privilegios.

32. Reestructuración de Base de Datos: Relaciones N:M (Juegos, Modalidades y Formatos) Objetivo: Refactorizar el esquema de la base de datos (DDL) para soportar la creación dinámica de torneos, permitiendo que un mismo juego soporte múltiples configuraciones (Ej: LoL puede jugarse 5v5 o 1v1, y en formato Liga o Suizo).

Normalización de Modalidades: Se eliminó el campo restrictivo `formato_equipo_defecto` de la tabla `juegos`. En su lugar, se creó la tabla puente `juego_modalidades_permitidas`. Esto permite una relación de Muchos a Muchos (N:M), donde un juego puede tener habilitadas varias modalidades (1v1, 5v5), optimizando la integridad referencial. Normalización de Formatos de Competición: Se creó la tabla `formatos_torneo` para independizar las estrategias de cruce (`liga`, `eliminacion_directa`, `suizo`). Para vincularlos, se generó la tabla intermedia `juego_formatos_permitidos`. Al crear un torneo, el Controlador ahora consulta estas tablas puente para renderizar dinámicamente las opciones válidas (JSON) según el juego seleccionado, previniendo que el Organizador elija combinaciones inválidas.

33. Asociación Polimórfica: Sistema de Guardados (Favoritos) Objetivo: Diseñar un sistema universal de "Guardados" (Favoritos) para que el Jugador pueda almacenar tanto Torneos como Equipos que le resulten de interés, sin ensuciar la base de datos con tablas redundantes.

Diseño Polimórfico (DDL): Se creó una única tabla llamada `guardados`. En lugar de tener `torneos_guardados` y `equipos_guardados` por separado, se utilizó el patrón de Asociación Polimórfica mediante el campo `tipo` (restringido con un `CHECK (tipo IN ('torneo', 'equipo'))`) junto con un `referencia_id`. Esto permite que una sola fila referencie dinámicamente a cualquier entidad del sistema. Restricciones de Integridad: Se aplicó un índice compuesto único (`UNIQUE KEY unique_guardado (usuario_id, tipo,`

`referencia_id`) para asegurar, a nivel de base de datos, que un usuario no pueda guardar la misma entidad dos veces, ahorrando validaciones costosas en PHP.

34. Trazabilidad de Acciones: Solicitudes de Equipo Objetivo: Identificar con precisión el origen de las solicitudes de ingreso a un equipo para el módulo de Notificaciones y Solicitudes.

Actualización Estructural: Se añadió el campo `iniciado_por` (Foreign Key hacia `usuarios`) en la tabla `solicitudes_equipo`. Esta modificación estructural permite al sistema distinguir si una solicitud pendiente fue enviada proactivamente por un Jugador (queriendo entrar al equipo), o si fue enviada como una invitación por el Capitán del Equipo (queriendo reclutar al jugador), bifurcando así la lógica de respuesta (Aceptar/Rechazar) en el Controlador.

35. Arquitectura Multigaming e Inferencia de Datos (Directorio de Equipos) Objetivo: Implementar un panel administrativo ("Directorio de Equipos") que permita listar todos los equipos del sistema, ordenarlos dinámicamente, y filtrarlos por disciplina (Juego) sin alterar el esquema relacional de la Base de Datos.

Manejo de Estructuras Multigaming: La tabla `equipos` fue diseñada para soportar organizaciones "Multigaming" (E-sports), por lo que no contiene la columna restrictiva `juego_id`. Un mismo equipo puede participar en torneos de distintos juegos simultáneamente. Inferencia de Filtros mediante JOINs: Para suplir el requerimiento administrativo de "Filtrar por Disciplina" sin romper el esquema Multigaming, el método `obtenerDirectorioAdmin()` en el modelo `Equipo.php` utiliza la técnica de "Inferencia por Participación". Mediante la cláusula `EXISTS`, el sistema busca si el equipo posee registros en la tabla `participantes_torneo` (cruzando con `torneos`) para la disciplina seleccionada. Esto preserva la normalización de la base de datos mientras cumple estrictamente los requisitos de interfaz de usuario. Agregación de Miembros (Subqueries): En la misma consulta del Directorio, se incluyó una subconsulta `(SELECT COUNT(*) FROM equipo_miembros...)` como columna virtual para calcular en tiempo real el tamaño de cada equipo, evitando redundancia de datos (desnormalización de contadores).

36. Autogestión de Perfil Administrativo (Seguridad en Interfaz) Objetivo: Habilitar la edición de datos básicos (Nombre y Teléfono) del Administrador General, blindando metadatos sensibles contra modificaciones accidentales o maliciosas.

Lógica de Exclusión Visual y Backend: En la vista de edición (`editar_perfil.php`), los campos críticos como `Correo Electrónico` (credencial de login) y `Rol` (Nivel de Permisos) se renderizan con el atributo `disabled` y estilos visuales de solo lectura. En el Backend, el método `actualizarPerfil()` del modelo `Usuario.php` está rigurosamente escrito para ejecutar el comando `UPDATE` afectando exclusivamente a las columnas `nombre_completo` y `telefono`, imposibilitando la escalada de privilegios o el secuestro de la cuenta (Account Takeover) a través del envío de peticiones POST manipuladas. Actualización de Sesión en Tiempo Real: Tras guardar los cambios exitosamente en la BD, el controlador sobreescribe la variable `$_SESSION['usuario_nombre']` para que la cabecera responsiva de navegación refleje el nuevo nombre instantáneamente sin requerir que el administrador vuelva a iniciar sesión.

37. Patrón "Read-Only Dashboard": Historial de Torneos Objetivo: Exhibir un registro auditable de los ciclos competitivos concluidos sin comprometer la base de datos de producción.

Filtro de Ciclo de Vida: El modelo `Torneo.php` fue dotado del método `obtenerHistorialTorneos()` el cual filtra drásticamente los registros con la cláusula `WHERE estado IN ('finalizado', 'cancelado')`. Esto garantiza que los torneos activos nunca se crucen con los registros archivados, protegiendo la integridad de la interfaz administrativa de gestión viva.

38. Integridad Analítica: Salón de la Fama Dinámico (Resultados Globales) Objetivo: Proveer un Ranking (Top 10) de los mejores jugadores y equipos históricos, resolviendo el desafío arquitectónico que supone cruzar puntuaciones de juegos con sistemas de medición drásticamente distintos (ej. Goles vs Rondas).

Descarte del Modelo "One-Size-Fits-All": Originalmente se pensó en un único Top 10 global. Se detectó que esto corrompería las métricas al sumar puntos de sistemas heterogéneos. Solución Dinámica Iterativa: En el `ResultadoGlobalControlador.php`, se programó un enfoque dinámico. Primero se invoca `obtenerJuegosConResultados()` para determinar qué disciplinas (juegos) tienen competencias finalizadas. Posteriormente, a través de un bucle `foreach`, se consulta individualmente el Top 10 específico por cada juego (`obtenerTop10PorJuego($juegoId)`). La vista (`globales.php`) recibe este arreglo multidimensional e itera renderizando una tabla independiente para cada Salón de la Fama, preservando la integridad competitiva al 100%.

39. Pureza Arquitectónica (MVC) y Desacoplamiento de UI: Data Attributes vs Inline Scripts Objetivo: Erradicar la mezcla de lógica de servidor (PHP) con interactividad de cliente (JS) en las vistas de presentación, purificando la adhesión al patrón Modelo-Vista-Controlador.

Refactorización de Flash Messages: Originalmente, las notificaciones emergentes (Toasts) se disparaban imprimiendo una etiqueta `<script>` vía PHP al pie de `dashboard.php`. Esto rompía la responsabilidad del Controlador y ensuciaba el DOM. Implementación de Data-Attributes: La vista maestra `dashboard.php` fue sanitizada. Ahora, PHP únicamente inyecta silenciosamente los metadatos en la etiqueta Body: `<body data-flash-mensaje="..." data-flash-tipo="...">`. Escucha Centralizada: El archivo estático `admin.js` fue dotado de un "Event Listener" global que se ejecuta tras el `DOMContentLoaded`. Este algoritmo lee nativamente el DOM y dispara las notificaciones Toast de forma autónoma. Con esta refactorización, se logra un 100% de aislamiento técnico entre PHP y JS para la lógica de negocios.

40. Auditoría Avanzada y Reforzamiento de Seguridad Perimetral (Fase 3) Objetivo: Blindar la aplicación a nivel corporativo mitigando vulnerabilidades críticas detectadas (OWASP Top 10) relativas a sesiones, inyección cruzada y fuerza bruta.

Seguridad de Sesiones (Timeboxing y Flags): Se actualizó la clase `Sesion.php` inyectando `session_set_cookie_params()` antes del inicio de sesión. Esto asegura que la cookie de identidad viaje protegida con los flags `HttpOnly` (inaccesible para JavaScript, previniendo secuestros XSS) y `SameSite=Lax`. Adicionalmente, se programó un motor de validación estricta de caducidad (Timeboxing) que expulsa al usuario del servidor si `$_SESSION['expiracion_sesion']` es superado, destruyendo el mito de las sesiones "infinitas". Políticas HTTP de Seguridad (Security Headers): Se creó el módulo núcleo `CabecerasSeguridad.php` y se inyectó en el Front Controller (`index.php`). Desde este momento, cualquier respuesta enviada al navegador incluye directivas estrictas (`X-Frame-Options: SAMEORIGIN`, `X-Content-Type-Options: nosniff`), protegiendo a la plataforma contra Clickjacking y Sniffing de manera nativa. Protección de Fuerza Bruta (Rate Limiting Profesional): Se desestimó la protección endeble basada en sesiones a favor de un bloqueo riguroso por Base de Datos. Se estructuró la tabla `intentos_login` administrada por el nuevo modelo `SeguridadAuth.php`. El Controlador de Autenticación ahora interroga a este modelo; si una IP registra 5 fallos consecutivos, es bloqueada automáticamente a nivel lógico por 5 minutos, garantizando la integridad de los accesos. Inmunidad Transaccional (Tokens CSRF): Se implementó el patrón *Synchronizer Token* en `Sesion.php`. Las áreas sensibles de Autenticación y el panel del Administrador General exigen ahora que los formularios envíen un `csrf_token` oculto validado por el servidor, haciendo matemáticamente imposible la falsificación de peticiones en nombre del usuario legítimo.

41. Implementacion detallada de proteccion CSRF en formularios de jugador Objetivo: Evitar que un sitio externo pueda enviar acciones POST en nombre de un usuario que ya tiene una sesion abierta en SGDM.

Patron utilizado: Se aplico el patron Synchronizer Token. El servidor genera un valor aleatorio por sesion y la vista lo incluye como campo oculto dentro de cada formulario POST:

```
<input type="hidden"
       name="csrf_token"
       value="<?php echo
htmlspecialchars(\App\Ayudantes\Sesion::generarTokenCSRF(), ENT_QUOTES, 'UTF-8');
?>">
```

Flujo del token:

1. `Sesion::generarTokenCSRF()` inicia la sesion y, si no existe un token, genera 32 bytes aleatorios con `random_bytes(32)`. El resultado se representa en hexadecimal mediante `bin2hex()` y se guarda en `$_SESSION['csrf_token']`.
2. La vista imprime el token en un campo oculto. El usuario no lo ve, pero el navegador lo envia con el formulario.
3. El controlador recibe el valor en `$_POST['csrf_token']`.
4. `Sesion::requerirTokensCSRF()` llama a `validarTokenCSRF()` y compara el token recibido con el token de la sesion mediante `hash_equals()`.
5. Si la comparacion falla, se responde con HTTP 419 y se detiene la accion antes de leer o modificar datos.

Decisiones de implementacion:

- El campo usa el nombre fijo `csrf_token` para que todas las vistas y controladores compartan el mismo contrato.
- Se usa `htmlspecialchars(..., ENT_QUOTES, 'UTF-8')` al imprimir el valor dentro de HTML.
- Cada formulario POST tiene su propio campo porque el navegador solo envia los controles que estan dentro del formulario enviado.
- La proteccion se coloca despues de comprobar que el metodo HTTP es POST y antes de procesar los datos recibidos.
- CSRF no sustituye la autenticacion ni la autorizacion. El controlador sigue validando que el usuario este logueado y tenga permisos.

Aplicacion en equipos:

- `crear-equipo.php`: formulario de creacion de equipo.
- `gestionar-equipo.php`: actualizacion, designacion de capitan, expulsion, solicitudes y salida del equipo.
- `EquipoControlador.php`: valida el token con `Sesion::requerirTokensCSRF()` en cada accion POST sensible.

Justificacion para la defensa: El token vincula el formulario con la sesion legitima. Un atacante puede intentar provocar una peticion desde otro sitio, pero no puede leer el token aleatorio de la sesion. Por eso la peticion

falsa es rechazada antes de ejecutar cambios.

42. Proteccion CSRF en acciones del panel del organizador Objetivo: Asegurar que las acciones POST que modifican torneos solo se ejecuten mediante formularios legitimos de la sesion activa.

En `PanelOrganizadorControlador` se distinguieron dos casos:

- `guardarPerfil()`, `guardarGestion()` y `guardarEtapa()` ya comprobaban que el metodo HTTP fuera POST. Se mantuvo esa validacion y se dejo `Sesion::requerirTokensCSRF()` inmediatamente despues.
- `cambiarEstado()`, `responderInscripcion()`, `generarFixture()` y `guardarResultado()` no comprobaban el metodo HTTP. Se agrego primero una respuesta HTTP 405 para cualquier metodo distinto de POST y luego la validacion CSRF.

El orden aplicado es intencional:

```
if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    http_response_code(405);
    exit('Metodo no permitido');
}

Sesion::requerirTokensCSRF();

// ReciÃ©n despues se leen $_POST y se llama al modelo.
```

Esto evita procesar datos enviados por GET y evita procesar datos POST sin un token asociado a la sesion. La proteccion no reemplaza `Sesion::validarOrganizador` o `Sesion::validarOrganizadorVerificado()`: esas validaciones siguen determinando quien esta autenticado y autorizado; el CSRF verifica que la peticion provenga de un formulario generado para esa sesion.

Se verifico la sintaxis de `panelOrganizadorControlador.php` con `php -l` y no se detectaron errores.

Justificacion para la defensa: Las acciones de cambiar estados, responder inscripciones, generar fixtures y guardar resultados modifican informacion del torneo. Por eso no deben aceptar GET y deben exigir el token CSRF antes de llamar al modelo o modificar la base de datos.

43. CSRF en la vista de gestion de torneos Objetivo: Alinear todos los formularios POST de gestion de torneos con las validaciones del controlador.

La vista gestionar-torneo.php contiene seis formularios POST:

- Guardar la configuracion general del torneo.
- Cambiar el estado del torneo.
- Aprobar una inscripcion.
- Rechazar una inscripcion.
- Generar o regenerar el fixture.
- Guardar el resultado de un partido.

Cada formulario incluye su propio campo oculto csrf_token. Los formularios de aprobacion y rechazo se generan dentro de un bucle, pero cada boton envia un formulario diferente; por eso cada formulario recibe un

token independiente. El formulario de resultado tambien puede repetirse por cada partido y debe mantener el token dentro de cada instancia.

La vista y el controlador deben estar alineados:

1. La vista envia csrf_token dentro del formulario.
2. El controlador rechaza metodos diferentes de POST.
3. El controlador valida el token mediante Sesion::requerirTokensCSRF().
4. Solo despues se leen los datos y se ejecuta la operacion del modelo.

Justificacion para la defensa: Se protege cada accion que cambia el torneo, incluyendo acciones repetidas dentro de bucles. Asi no queda un formulario administrativo sin la misma garantia de autenticidad de la sesion.

44. Acciones administrativas de organizadores: migracion de GET a POST Objetivo: Evitar que aprobar, rechazar o revocar la habilitacion de un organizador pueda ejecutarse mediante un enlace GET.

Antes del cambio, la vista organizador_index.php utilizaba enlaces con id y decision en la URL. El controlador leia esos valores desde $_GET y ejecutaba una modificacion de estado. Se reemplazaron esas acciones por formularios POST.

El controlador OrganizadorControlador ahora:

1. Rechaza metodos distintos de POST con HTTP 405.
2. Valida el token de la sesion mediante Sesion::requerirTokensCSRF().
3. Lee id y decision desde $_POST.
4. Acepta unicamente las decisiones aprobar, rechazar y revocar.
5. Ejecuta el modelo y redirecciona al listado.

La vista genera un formulario independiente para cada accion y envia tres valores: csrf_token, id y decision. El boton visible ya no es un enlace que modifica datos, sino un submit POST.

Justificacion para la defensa: Aprobar o revocar un organizador cambia sus permisos dentro del sistema. Las acciones con efectos secundarios no deben ejecutarse por GET, porque un navegador puede disparar un enlace sin una confirmacion explicita. POST mas CSRF obliga a enviar una peticion intencional y vinculada a la sesion del administrador.

Las acciones verComo, editarUsuario y cambiarEstadoUsuario quedan fuera de este cambio y deben revisarse en sus respectivos controladores.

45. Proteccion de la eliminacion de usuarios Objetivo: Evitar que la baja fisica de un usuario pueda ejecutarse mediante un enlace GET o una peticion falsificada.

Antes, eliminarUsuario() leia id y rol_id desde $_GET. Esto significaba que una URL podia provocar una accion destructiva si el administrador tenia una sesion abierta. La vista jugadores_index.php usaba un enlace para eliminar la cuenta.

El cambio aplicado conserva la logica del modelo, pero cambia el contrato HTTP:

1. Solo se acepta el metodo POST.
2. Se exige una sesion de administrador.

3. Se valida el token CSRF.
4. id y rol_id se leen desde $_POST.
5. El modelo recibe el mismo id que antes.

La vista ahora utiliza un formulario POST con un boton y una confirmacion JavaScript. El formulario envia csrf_token, id y rol_id. rol_id no identifica la seguridad de la operacion; se conserva para que el controlador redirija al listado de jugadores u organizadores correspondiente despues de la operacion.

La confirmacion visual no reemplaza las validaciones del servidor. La proteccion real esta en la combinacion de autorizacion de administrador, metodo POST y token CSRF.

No se elimino ningun usuario durante este cambio. Se validaron la sintaxis del controlador y de la vista con php -l.

46. Entregables DevOps para despliegue y defensa Objetivo: Preparar un procedimiento repetible para levantar ASCEND en una VM Linux dentro de Proxmox, operar Docker, conservar logs y recuperar la base.

Se agregaron tres documentos operativos:

- Guia_Despliegue_10_Minutos.md: secuencia rapida de preparacion, copia, validacion, build, arranque y prueba HTTP.
- Guia_DevOps_Servidor_Linux.md: responsabilidades del operador, usuarios, permisos, firewall, hora, actualizaciones y monitoreo.
- Guia_Backup_Restauracion_Logs.md: backup MySQL, checksum, retencion, restauracion controlada, logs y hashes de archivos PHP.

Los comandos destructivos se separaron de los comandos de operacion diaria. La restauracion y la eliminacion de volumenes exigen una decision explicita, un backup previo y una verificacion del objetivo.

Durante la validacion se detecto que el Compose actual no pasa la comprobacion porque la condicion de depends_on no esta expresada como un mapeo YAML. Este control se mantiene en la guia de despliegue para que la presentacion no continue con una configuracion invalida.

47. Auditoria real de conectividad y despliegue dual Objetivo: comprobar que ASCEND funciona tanto en XAMPP como en Docker y separar los errores de conexion de los problemas de esquema.

Pruebas realizadas:

- XAMPP MySQL responde con root y la base ascend existe.
- XAMPP responde HTTP 200 en la URL local.
- Docker Compose valida su configuracion.
- sgdm_db queda en estado healthy.
- El usuario de aplicacion ascend_usuario@% conecta con permisos CRUD limitados.
- sgdm_web responde HTTP 200 en el puerto 8080.
- phpMyAdmin y Grafana responden en sus puertos locales.
- Todos los archivos PHP pasan php -l.

Correcciones de conexion:

1. La configuracion PHP ahora lee DB_PASSWORD desde variables de entorno. En XAMPP el fallback sigue siendo contraseña vacia; en Docker se usa ascend_password.
2. Se corrigio depends_on para que web espere a que db este saludable.
3. Se corrigio el healthcheck para expandir MYSQL_ROOT_PASSWORD dentro del shell del contenedor.
4. Se creo el usuario Docker de aplicacion que faltaba en el volumen existente, sin modificar tablas ni datos.

Hallazgo de integracion:

XAMPP contiene 39 tablas, mientras que Docker contiene 26 y no tiene politicas_contrasenas. Por lo tanto, HTTP 200 y una conexion exitosa no significan que los dos entornos tengan el mismo esquema. La sincronizacion del esquema queda como una tarea separada y no debe resolverse ejecutando una migracion destructiva sobre datos existentes.

La comparacion de nombres identifico 13 tablas presentes en XAMPP y ausentes en Docker: auditoria_cambios, historial_contrasenas, logros, logs_actividad, newsletter_subscriptores, notificaciones, politicas_contrasenas, resultados_detalle, torneo_actividades, torneo_comentarios, torneo_config, torneo_suizo_parejas y usuario_logros.

48. Reinicializacion reproducible de la base Docker

Para eliminar la desincronizacion se genero primero un respaldo del volumen Docker en `Documentos_Juan/backups_docker_reset/ascend_docker_antes_reset.sql`. Su SHA-256 es `BDD805A84E7D0D20B210BF5A766EA4821B08C86B2EAD912AE23DDCD41D73DAAC`. Luego se detuvo el Compose y se elimino exclusivamente el volumen `docker_sgdm_datos`. La base de XAMPP no fue modificada.

El primer arranque desde cero revelo una incompatibilidad real de MySQL 8 en `base_de_datos/migraciones/001_ascend.sql`: el `CHECK distintos` de `torneo_suizo_parejas` usa columnas que tambien poseen claves foraneas con acciones referenciales. MySQL rechazo la tabla completa. Se corrigio el script de migracion retirando solamente ese `CHECK`; la validacion equivalente queda en el flujo de creacion de parejas del modelo `Torneo`.

Tras corregir la migracion, el segundo arranque ejecuto 001 y 002 correctamente. Docker quedo con 39 tablas, igual que XAMPP, `politicas_contrasenas` presente, usuario de aplicacion funcional y todos los servicios principales activos. La portada de Docker y XAMPP respondieron HTTP 200. Docker contiene usuarios de ejemplo por la migracion 002, mientras XAMPP no; esa diferencia es de datos iniciales y no de estructura.